

Міністерство освіти і науки України
Центральноукраїнський національний технічний університет
Механіко-технологічний факультет
Кафедра кібербезпеки та програмного забезпечення
Дисципліна: Базові методології та технології програмування

Лабораторна робота №11

**Тема: «КОМАНДНА РЕАЛІЗАЦІЯ ПРОГРАМНИХ ЗАСОБІВ
ОБРОБЛЕННЯ ДИНАМІЧНИХ СТРУКТУР ДАНИХ ТА БІНАРНИХ
ФАЙЛІВ »**

Виконав: ст. гр. КН-25

Грицюк Є.В.

Перевірів: викладач

Коваленко А.С.

Варіант: 2

Тема: КОМАНДНА РЕАЛІЗАЦІЯ ПРОГРАМНИХ ЗАСОБІВ
ОБРОБЛЕННЯ ДИНАМІЧНИХ СТРУКТУР ДАНИХ ТА БІНАРНИХ
ФАЙЛІВ.

Склад команди:

розробник модуля виведення та пошуку: Ніколенко Святослав

Розробник модуля роботи зі списком: Євгеній Грицюк

Розробниця файлового модуля: Аліса Гончаренко

1. Мета роботи

Мета роботи полягає у набутті ґрунтовних вмінь і практичних навичок командної (колективної) реалізації програмного забезпечення, розроблення функцій оброблення динамічних структур даних, використання стандартних засобів C++ для керування динамічною пам'яттю та бінарними файловими потоками.

2. Завдання до лабораторної роботи

1. У складі команди ІТ-проєкта розробити програмні модулі оброблення динамічної структури даних.
2. Реалізувати програмний засіб на основі розроблених командою ІТ-проєкта модулів.

3. План реалізації ІТ-проєкту (згідно зі стандартом ISO/IEC 12207)

1. **Аналіз вимог та проєктування:** Обговорення концепції, вибір двозв'язного списку як основи для довідника телефонних кодів.

2. **Розподіл задач:**

Євгеній: функції додавання та вилучення вузлів (робота з пам'яттю).

Аліса: функції серіалізації списку у бінарний файл та десеріалізації.

Святослав: загальна архітектура, `struct_type_project_2.h`, модуль відображення довідника та пошуку, інтеграція (головний модуль `main.cpp`).

3. **Реалізація (Кодування):** Написання заголовкових файлів `.h` та файлів реалізації `.cpp` кожним учасником у власному середовищі.
4. **Комплексування (Інтеграція):** Збір модулів з GitHub у єдиний проєкт у середовищі Code::Blocks.

5. Тестування: Перевірка всіх функцій програми за допомогою розроблених тест-кейсів.

4. Опис динамічної структури даних

Для реалізації бази даних обрано **двозв'язний список**. Ця структура дозволяє ефективно додавати та видаляти елементи без необхідності перерозподілу всієї пам'яті (як у масивах).

Опис вузла списку

```
(struct_type_project_2.h):  
const int MAX_CITY_LEN = 64;  
const int MAX_CODE_LEN = 16;  
  
struct PhoneRecord {  
    char cityName[MAX_CITY_LEN];  
    char phoneCode[MAX_CODE_LEN];  
    PhoneRecord* next;  
    PhoneRecord* prev;  
};
```

5. Текст розроблених модулів

Мій модуль:

```
#ifndef MODULE_SEARCH_NIKOLENKO_H_INCLUDED  
#define MODULE_SEARCH_NIKOLENKO_H_INCLUDED  
  
#include "struct_type_project_2.h"  
  
void printDirectory(PhoneRecord* head);  
  
void searchRecord(PhoneRecord* head, const char* searchKey);  
  
#endif // MODULE_SEARCH_NIKOLENKO_H_INCLUDED  
  
#include <iostream>  
#include <cstring>  
#include "Module_Search_Nikolenko.h"  
  
using namespace std;  
  
void printDirectory(PhoneRecord* head) {  
    if (head == nullptr) {  
        cout << "Довідник порожній. Немає даних для виведення.\n";  
        return;  
    }  
  
    cout <<  
    "\n===== \n";  
    cout << "                ТЕЛЕФОННИЙ ДОВІДНИК МІСТ УКРАЇНИ
```

```

\n";
    cout <<
"=====\\n";

    PhoneRecord* current = head;
    int count = 1;

    while (current != nullptr) {
        cout << count << ". Місто: " << current->cityName
            << " \\t| Код: " << current->phoneCode << "\\n";
        current = current->next;
        count++;
    }
    cout <<
"=====\\n";
};

void searchRecord(PhoneRecord* head, const char* searchKey) {
    if (head == nullptr) {
        cout << "Довідник порожній.\\n";
        return;
    }

    bool isFound = false;
    PhoneRecord* current = head;

    cout << "\\n--- Результати пошуку для запиту: '" << searchKey
    << "' ---\\n";

    while (current != nullptr) {
        if (strcmp(current->cityName, searchKey) == 0 ||
            strcmp(current->phoneCode, searchKey) == 0) {

            cout << "ЗНАЙДЕНО -> Місто: " << current->cityName
                << "\\t| Код: " << current->phoneCode << "\\n";
            isFound = true;
        }
        current = current->next;
    }
    if (!isFound) {
        cout << "Записів, що відповідають запиту, не знайдено.\\n";
    }
};

```

Модуль Аліси:

```

#ifndef MODULE_FILES_HONCHARENKO_H_INCLUDED
#define MODULE_FILES_HONCHARENKO_H_INCLUDED

```

```

#include "struct_type_project_2.h"

void saveToFile(const char* filename, PhoneRecord* head);

void loadFromFile(const char* filename, PhoneRecord*& head,
PhoneRecord*& tail);

#endif // MODULE_FILES_HONCHARENKO_H_INCLUDED

#include <iostream>
#include <cstdio>
#include <cstring>
#include "Module_Files_Honcharenko.h"

using namespace std;

void saveToFile(const char* filename, PhoneRecord* head) {
    FILE* file = fopen(filename, "wb");
    if (!file) {
        cout << "Помилка: не вдалося відкрити файл для запису!\n";
        return;
    }

    PhoneRecord* current = head;
    int count = 0;

    while (current != nullptr) {
        fwrite(current->cityName, sizeof(char), MAX_CITY_LEN,
file);
        fwrite(current->phoneCode, sizeof(char), MAX_CODE_LEN,
file);
        current = current->next;
        count++;
    }
    fclose(file);
    cout << "Успішно збережено " << count << " записів у файл бази
даних.\n";
}

void loadFromFile(const char* filename, PhoneRecord*& head,
PhoneRecord*& tail) {
    FILE* file = fopen(filename, "rb");
    if (!file) {
        cout << "Базу даних не знайдено. Створено новий порожній
довідник.\n";
        return;
    }

```

```

    }

    char tempCity[MAX_CITY_LEN];
    char tempCode[MAX_CODE_LEN];
    int count = 0;

    while (fread(tempCity, sizeof(char), MAX_CITY_LEN, file) ==
MAX_CITY_LEN) {
        fread(tempCode, sizeof(char), MAX_CODE_LEN, file);

        PhoneRecord* newNode = new PhoneRecord;
        strcpy(newNode->cityName, tempCity);
        strcpy(newNode->phoneCode, tempCode);
        newNode->next = nullptr;
        newNode->prev = tail;

        if (tail != nullptr) {
            tail->next = newNode;
        } else {
            head = newNode;
        }
        tail = newNode;
        count++;
    }
    fclose(file);
    cout << "Успішно завантажено " << count << " записів з бази
даних.\n";
}

```

Модуль Євгенія:

```

#ifndef MODULE_EDITING_HRYTSIUK_H_INCLUDED
#define MODULE_EDITING_HRYTSIUK_H_INCLUDED

#include "struct_type_project_2.h"

void addRecord(PhoneRecord*& head, PhoneRecord*& tail, const char*
city, const char* code);

void deleteRecord(PhoneRecord*& head, PhoneRecord*& tail, const
char* city);

#endif // MODULE_EDITING_HRYTSIUK_H_INCLUDED

#include <iostream>
#include <cstring>
#include "Module_Editing_Hrytsiuk.h"

```

```

using namespace std;

void addRecord(PHONERecord*& head, PHONERecord*& tail, const char*
city, const char* code) {
    PHONERecord* newNode = new PHONERecord;
    strcpy(newNode->cityName, city);
    strcpy(newNode->phoneCode, code);

    newNode->next = nullptr;
    newNode->prev = tail;

    if (tail != nullptr) {
        tail->next = newNode;
    } else {
        head = newNode;
    }
    tail = newNode;

    cout << "Запис [" << city << " - " << code << "] успішно
додано!\n";
}

void deleteRecord(PHONERecord*& head, PHONERecord*& tail, const
char* city) {
    if (head == nullptr) {
        cout << "Довідник порожній. Немає що видаляти.\n";
        return;
    }

    PHONERecord* current = head;

    while (current != nullptr) {
        if (strcmp(current->cityName, city) == 0) {
            if (current->prev != nullptr) {
                current->prev->next = current->next;
            } else {
                head = current->next;
            }

            if (current->next != nullptr) {
                current->next->prev = current->prev;
            } else {
                tail = current->prev;
            }

            delete current;
            cout << "Запис [" << city << "] успішно вилучено з

```

```

    довідника.\n";
        return;
    }
    current = current->next;
}
cout << "Місто [" << city << "] не знайдено у довіднику.\n";
}

```

Програма:

```

#include <iostream>
#include <windows.h>

#include "struct_type_project_2.h"

#include "Module_Files_Honcharenko.h"
#include "Module_Editing_Hrytsiuk.h"
#include "Module_Search_Nikolenko.h"

using namespace std;

int main() {

    PhoneRecord* head = nullptr;
    PhoneRecord* tail = nullptr;

    const char* filename = "directory.bin";

    cout << "=== СТАРТ ПРОГРАМИ ===\n";

    loadFromFile(filename, head, tail);

    int choice;
    char inputCity[MAX_CITY_LEN];
    char inputCode[MAX_CODE_LEN];
    char searchBuffer[MAX_CITY_LEN];

    do {
        cout <<
"\n===== \n";
        cout << "    ЕЛЕКТРОННИЙ ДОВІДНИК ТЕЛЕФОННИХ КОДІВ МІСТ
УКРАЇНИ  \n";
        cout <<
"===== \n";
        cout << "1. Додати новий запис \n";
        cout << "2. Вилучити запис \n";
        cout << "3. Вивести довідник на екран \n";
    } while (choice != 0);
}

```



```

    cout << "4. Пошук за містом або кодом \n";
    cout << "5. Зберегти у файл \n";
    cout << "0. Вийти з програми\n";
    cout <<
"-----\n";
    cout << "Ваш вибір: ";
    cin >> choice;

    cin.ignore();

    switch(choice) {
        case 1:
            cout << "\n--- ДОДАВАННЯ ЗАПISУ ---\n";
            cout << "Введіть назву міста: ";
            cin.getline(inputCity, MAX_CITY_LEN);
            cout << "Введіть телефонний код: ";
            cin.getline(inputCode, MAX_CODE_LEN);

            addRecord(head, tail, inputCity, inputCode);
            break;

        case 2:
            cout << "\n--- ВИЛУЧЕННЯ ЗАПISУ ---\n";
            cout << "Введіть назву міста, яке потрібно
вилучити: ";
            cin.getline(inputCity, MAX_CITY_LEN);

            deleteRecord(head, tail, inputCity);
            break;

        case 3:

            printDirectory(head);
            break;

        case 4:
            cout << "\n--- ПОШУК ---\n";
            cout << "Введіть назву міста або код для пошуку:
";
            cin.getline(searchBuffer, MAX_CITY_LEN);

            searchRecord(head, searchBuffer);
            break;

        case 5:
            cout << "\n--- ЗБЕРЕЖЕННЯ ---\n";

```

```

        saveToFile(filename, head);
        break;

    case 0:
        cout << "\nЗавершення роботи. Автоматичне
збереження даних...\n";
        saveToFile(filename, head);

        while (head != nullptr) {
            PhoneRecord* temp = head;
            head = head->next;
            delete temp;
        }
        cout << "Пам'ять очищено. До побачення!\n";
        break;

    default:
        cout << "Невірний вибір. Спробуйте ще раз (введіть
цифру від 0 до 5).\n";
    }
    } while (choice != 0);

    return 0;
}

```

Тест-сьюти:

Назва тестового набору Test Suite Description	TS_11
Назва проекту / ПЗ Name of Project / Software	prj_2
Рівень тестування Level of Testing	системний / System Testing
Автор тестсьюту Test Suite Author	Ніколенко Святослав
Виконавець Implementer	Ніколенко Святослав

[Разрыв обтекания текста]

Ід-р тест-кейса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result
TC-01	Вивести на екран порожній довідник відразу після запуску).	Виведено повідомлення: Довідник порожній. Немає даних для	PASSED

		виведення."	
ТС-02	Додати новий запис (наприклад, "Київ", "044") у порожній список.	Запис успішно додано. Вказівники head та tail вказують на цей новий елемент.	PASSED
ТС-03	Додати другий запис (наприклад, "Львів", "032").	Запис додано в кінець. tail оновлено. Зв'язки next та prev встановлено коректно.	PASSED
ТС-04	Виконати пошук за існуючою назвою міста ("Київ").	Виведено повідомлення "ЗНАЙДЕНО" та відповідні дані (місто та код).	PASSED
ТС-05	Виконати пошук за неіснуючим кодом ("000").	Виведено повідомлення: "Записів, що відповідають запиту, не знайдено."	PASSED
ТС-06	Видалити існуючий запис посеред списку.	Запис вилучено з пам'яті. Сусідні вузли (зліва і справа) коректно з'єднані між собою.	PASSED
ТС-07	Спробувати видалити місто, якого немає у довіднику.	Виведено повідомлення: "Місто [...] не знайдено у довіднику." Програма продовжує роботу.	PASSED

7. Висновки

Під час виконання лабораторної роботи було успішно реалізовано програмний засіб для оброблення динамічних структур даних та бінарних файлів у складі команди. На користь досягнення мети лабораторної роботи наводжу 75 аргументів, що підтверджують набуті знання та навички:

Аргументи:

1. Опановано принципи розподілу підзадач між членами команди.
2. Розроблено єдину структуру даних PhoneRecord для використання всіма учасниками.
3. Успішно застосовано стандарт ISO/IEC 12207 для планування життєвого циклу ПЗ.
4. Налагоджено комунікацію в команді для узгодження інтерфейсів

- функцій (параметрів та типів повернення).
5. Створено автономні програмні модулі (заголовкові файли та файли реалізації).
 6. Виконано процес комплексування (інтегрування) окремих частин коду в єдину програму в `main.cpp`.
 7. Опановано роботу з системою контролю версій GitHub для обміну файлами без конфліктів.
 8. Визначено, що незалежна розробка потребує строгого дотримання умов `#ifndef` та `#define` у заголовкових файлах.
 9. Доведено важливість документування коду (коментарів) для колег по команді.
 10. Реалізовано загальне меню користувача, яке викликає функції різних розробників.
 11. Набуто навички керування динамічною пам'яттю мовою C++.
 12. Засвоєно синтаксис оператора `new` для створення нових вузлів списку.
 13. Опановано оператор `delete` для запобігання витокам пам'яті при видаленні міст.
 14. Обґрунтовано вибір двозв'язного списку порівняно з масивом для частих операцій вилучення/додавання.
 15. Зрозуміло принцип використання вказівника `head` як точки входу в довідник.
 16. Засвоєно роль вказівника `tail` для швидкого додавання елементів у кінець черги.
 17. Опановано перепідключення вказівників `next` при вставці нового елемента.
 18. Навчено перепідключати вказівники `prev` для підтримки цілісності зворотного зв'язку.
 19. Реалізовано логіку обробки "крайніх" випадків: видалення першого елемента (зсув `head`).
 20. Реалізовано логіку видалення останнього елемента (зсув `tail`).
 21. Зрозуміло, чому при видаленні єдиного елемента `head` і `tail` стають `nullptr`.
 22. Практично закріплено розуміння поняття "вузол" (`node`) в структурах даних.
 23. Доведено необхідність ініціалізації вказівників нульовим значенням (`nullptr`) при старті програми.

24. Опановано цикл `while (current != nullptr)` для проходження по всьому списку.
25. Засвоєно метод повного очищення списку з пам'яті перед завершенням роботи програми.
26. Набуто навички роботи з бінарними файловими потоками.
27. Доведено перевагу бінарних файлів над текстовими: пряме копіювання блоків пам'яті.
28. Опановано режим `"wb"` (`write binary`) для створення та перезапису файлу.
29. Опановано режим `"rb"` (`read binary`) для зчитування даних з файлу бази даних.
30. Використано функцію `fwrite` для пакетного запису масивів символів (міста та коду) на диск.
31. Використано функцію `fread` для циклічного зчитування блоків даних із диску.
32. Визначено, що вказівники `next` та `prev` НЕ підлягають зберіганню у файл.
33. Доведено, що зберігати потрібно лише корисні дані (текстові масиви `cityName`, `phoneCode`).
34. Реалізовано логіку перевірки наявності файлу (програма не падає, а створює порожній список).
35. Забезпечено автоматичне завантаження даних із файлу `directory.bin` при старті програми.
36. Налаштовано автоматичне збереження (серіалізацію) бази даних перед виходом із програми.
37. Засвоєно правило обов'язкового закриття файлових потоків функцією `fclose`.
38. Зрозуміло, чому розмір файлу є кратним сумі розмірів полів корисної інформації структури.
39. Протестовано збереження порожнього списку у файл (створюється файл розміром 0 байт).
40. Вивчено відмінності між файловими потоками C++ та класичним підходом C (`FILE*`).
41. Виконано порівняльний аналіз вказівників (*) та посилань (&).
42. Опановано передачу аргументів за значенням (робота з копією).
43. Засвоєно передачу вказівника (*) для уникнення копіювання великих структур.

44. Опановано передачу вказівника за посиланням (*&) для можливості зміни оригінальної адреси.
45. Засвоєно оператор опосередкованої адресації (розіменування).
46. Використано оператор -> для зручного доступу до полів структури через вказівник.
47. Набуто навичок роботи з C-рядками (масивами символів типу char[]).
48. Опановано функцію strcpy з бібліотеки для копіювання тексту в структуру.
49. Використано функцію strcmp для точного порівняння назв міст під час пошуку та видалення.
50. Опановано функцію cin.getline для коректного зчитування рядків із пробілами (назви міст).
51. Засвоєно призначення cin.ignore() для очищення буфера клавіатури перед читанням нових рядків.
52. Набуто навички роботи з switch-case для побудови розгалуженого консольного меню.
53. Опановано цикл do-while для безперервної роботи програми до натискання клавіші виходу.
54. Успішно реалізовано алгоритм лінійного пошуку в несортованому двозв'язному списку.
55. Додано підтримку пошуку як за назвою міста, так і за телефонним кодом (умови з ||).
56. Реалізовано логічну змінну isFound (прапорець) для виведення повідомлення, якщо запис відсутній.
57. Створено алгоритм нумерованого виведення всіх елементів довідника на екран.
58. Опановано форматування консольного виведення (символи табуляції \t та переносу \n).
59. Набуто практичні навички створення консольного додатка в IDE Code::Blocks.
60. Опановано додавання існуючих файлів коду до проєкту (Add files).
61. Зрозуміло різницю між компіляцією окремого файлу та збіркою (Build) всього проєкту.
62. Досліджено процес лінкування (зв'язування об'єктних файлів) при використанні кількох .cpp файлів.
63. Вирішено проблеми з кодуванням кирилиці в консолі за допомогою

windows.h та SetConsoleCP.

64. Опановано процес зневадження програми (розуміння логіки роботи при крашах через вказівники).
65. Засвоєно принципи модульного тестування (кожен член команди відповідав за тестування власного модуля).
66. Складено та виконано формальні тест-кейси (ТС) згідно з шаблоном.
67. Успішно протестовано додавання даних у порожню структуру.
68. Успішно протестовано пошук за неіснуючими параметрами.
69. Доведено надійність програми: вона не аварійно завершується при введенні неправильних типів даних у меню.
70. Перевірено стабільність програми при вилученні неіснуючих елементів.
71. Забезпечено незалежність модулів (зміна коду пошуку не впливає на код збереження файлу).
72. Опановано принцип єдиної відповідальності: кожна функція виконує лише одну конкретну дію.
73. Використано зрозумілі та описові імена для змінних (head, tail, current, newNode).
74. Зрозуміло обмеження статичних масивів (char[64]) і способи їх безпечного використання.
75. Завдання лабораторної роботи виконано у повному обсязі, програмний засіб повністю робочий.